

RESEARCH ARTICLE

A Multi-Stage Backdoor Detection (MSBD) Framework

HOSSEIN ABROSHAN^{ID}, (Senior Member, IEEE), AND SYED WAQUAS HASHMI^{ID}

School of Computing and Information Sciences, Anglia Ruskin University, CB1 1PT Cambridge, U.K.

Corresponding author: Hossein Abroshan (hossein.abroshan@aru.ac.uk)

This work was supported by APC funded by the Anglia Ruskin University.

ABSTRACT Backdoor attacks represent a serious challenge to robust deployment of machine learning (ML) and deep learning (DL) models in safety- and mission-critical fields. In a backdoor attack, an adversary injects a hidden trigger so that the model behaves normally when the inputs are clean but consistently produces attacker-chosen outputs when the trigger is present. Existing defences generally work at a single stage in the ML lifecycle—on data, on the model, or at inference time—and are thus susceptible to adaptive attackers that intentionally evade their underlying assumptions. This paper proposes *Multi-Stage Backdoor Detection (MSBD)*, which provides a defence-in-depth structure that combines multiple mechanisms in the training, post-training model inspection, and deployment-time monitoring. MSBD has four stages: influence-based screening of training samples (Stage A), optimisation-based trigger inversion (Stage B), neuron activation graph analysis for detection of suspicious subnetworks (Stage C), as well as calibrated runtime detection with integration of trigger signatures and perturbation-based consistency checks (Stage D). The platform is intended to function under realistic defender conditions with limited access to both data and models, and for offline validation and online monitoring. We evaluate MSBD on three benchmark datasets (MNIST, CIFAR-10, GTSRB) under a strong BadNets-style backdoor attack and compare it against five representative defences (STRIP, Neural Cleanse, Activation Clustering, Spectral Signatures, and Fine-Pruning). Across all datasets, we find an average F1-score of 99.0% for MSBD, which is consistently better than STRIP's, with a practical runtime overhead, showing that multi-stage, cross-layer defences can significantly improve robustness over single-stage defences.

INDEX TERMS Backdoor attacks, trojan attacks, adversarial machine learning, runtime detection, model inspection, critical infrastructure security.

I. INTRODUCTION

Machine learning (ML) and deep learning (DL) models are increasingly being used in decision-critical systems on critical infrastructure, including industrial control systems (ICS), smart grids, intelligent transportation, and healthcare, where high-fidelity decisions may mean the difference between normal operation and catastrophic failure. However, such integration exposes these systems to a growing spectrum of adversarial threats with potentially severe safety and economic consequences.

Adversarial manipulation of ML models is commonly categorised into three broad dimensions: evasion attacks,

which perturb inputs at inference time without modifying the model itself [1], [2]; poisoning or backdoor attacks, where the training data or training process is manipulated so that the model learns a hidden trigger-based behaviour [3]; and model extraction or compromise, where an adversary attempts to infer model parameters or internal logic via queries [4]. Among these, backdoor poisoning attacks are particularly pernicious in safety-critical applications. An attacker implants a hidden *trigger* so that the model behaves normally on clean inputs but consistently outputs attacker-specified predictions whenever the trigger appears [5]. Because backdoored models typically maintain high accuracy on standard validation sets, they can easily evade conventional testing and propagate through ML supply chains and third-party model distribution channels.

The associate editor coordinating the review of this manuscript and approving it for publication was Tiago Cruz^{ID}.

The stealthiness and specificity of backdoor attacks have motivated a large body of defensive research. Data-centric defences, such as Activation Clustering and Spectral Signatures, attempt to identify poisoned samples via anomalies in feature space [6], [7]. Model-centric defences, exemplified by Neural Cleanse and Artificial Brain Stimulation, reverse engineer potential triggers or identify suspicious neurons responsible for malicious behaviour [8], [9]. Runtime defences, such as STRIP and SentiNet, monitor inference-time behaviour to flag inputs that appear to contain triggers [10], [11]. While each category is effective in certain scenarios, most existing defences are designed for a specific stage of the ML lifecycle and often rely on strong assumptions about the trigger (e.g., static, visible, or universal). As a result, individual defences can be circumvented by adaptive attackers that deliberately violate these assumptions, for example via clean-label attacks [12], input-aware or dynamic triggers [13], or distributed/backdoor representations.

To address these gaps, we propose *Multi-Stage Backdoor Detection (MSBD)*, a defence-in-depth framework that integrates complementary detection mechanisms across multiple stages of the ML lifecycle. MSBD combines influence-based training data inspection, generator-based trigger inversion, graph-based neuron activation analysis, and calibrated runtime monitoring. This multi-pronged architecture raises the difficulty for adversaries, as successful evasion would require simultaneously defeating orthogonal detection mechanisms. Moreover, MSBD is designed with operational constraints of critical infrastructure in mind, enabling integration into existing model development workflows and deployment with lightweight runtime overhead.

This study investigates whether a defence-in-depth design can provide stronger and more reliable backdoor detection than single-stage defences under a standard but strong patch-based backdoor threat model. Specifically, we address the following research questions: RQ1) Does the proposed Multi-Stage Backdoor Detection (MSBD) framework outperform representative data-centric, model-centric, and runtime backdoor defences across multiple benchmark datasets? RQ2) How robust is MSBD to variations in trigger location and intensity relative to the configuration observed during training? Accordingly, our objectives are to design an end-to-end multi-stage detection pipeline spanning training-time screening, model-level inspection, and runtime monitoring, and to empirically evaluate its effectiveness and robustness against well-established backdoor attacks.

The main contributions of this work are summarised as follows:

- Proposes MSBD, a defence-in-depth framework that integrates training-time screening, post-training model inspection, and runtime monitoring for backdoor detection.
- Introduces a multi-head trigger-inversion stage that recovers compact trigger candidates to guide downstream analysis.

- Proposes neuron activation graph analysis (with modularity-based scoring) to localise suspicious Trojan subnetworks.
- Evaluates MSBD on MNIST, CIFAR-10, and GTSRB under strong BadNets-style patch backdoors, and compares against five representative baselines, showing consistent gains over STRIP.
- Reports ablation, false-positive impact (Stage A), and per-stage timing results to support practical deployability in safety-critical settings.

The rest of the paper is organised as follows. Section II reviews ML security threats, backdoor attack models, and existing defence strategies. Section III presents the MSBD architecture and details each stage. Section IV reports an extensive evaluation on three benchmark datasets and compares MSBD to state-of-the-art defences. Section V discusses implications, limitations, and deployment considerations, particularly in the context of critical infrastructures. Section VI concludes and outlines directions for future work.

II. BACKGROUND

A. MACHINE LEARNING SECURITY THREATS

Modern ML systems are vulnerable to a range of adversarial threats that can be broadly categorised into evasion attacks, poisoning attacks, and model tampering attacks [1]. Evasion attacks (adversarial examples) perturb test-time inputs to cause misclassification without modifying the model or training data [14]. Poisoning attacks modify the training data or process to influence the learned model, potentially degrading performance or embedding targeted behaviours [15]. Model tampering or Trojan attacks directly alter model parameters or training pipelines to implant concealed behaviours [1]. In security-critical domains, each threat class presents serious risks to safe and trustworthy ML deployment [16].

B. BACKDOOR ATTACKS: THREAT MODEL AND STEALTHINESS

Backdoor (Trojan) attacks are a particularly severe form of training-time attack [17]. An adversary aims to inject a hidden trigger or pattern into the model so that it behaves normally on clean inputs, but reliably misclassifies inputs containing the trigger to an attacker-chosen target class [18], [19]. Triggers can be small image patches, stickers, or even imperceptible perturbations. Because the backdoor is rarely activated during standard validation, backdoored models can exhibit near-normal accuracy on clean test data while harbouring latent malicious behaviour [7].

Backdoor attacks are especially relevant in ML supply chains, where data collection, training, and deployment may involve untrusted third parties [8], [10]. Typical threat scenarios include poisoned third-party datasets, compromised outsourced training services, trojaned pre-trained models from model zoos, and post-training tampering on untrusted deployment platforms [11], [20]. Early work such as BadNets demonstrated that small patch triggers could cause targeted

misclassification in traffic sign recognition models without degrading clean accuracy [5], [19]. Subsequent research extended these attacks to clean-label settings, invisible or blended triggers, and dynamic or input-aware triggers that further challenge existing defences [12], [13], [21], [22].

C. BACKDOOR DEFENCE STRATEGIES

Backdoor defences can be grouped into three broad categories: data-driven, model-driven, and runtime defences.

Data-driven defences inspect training data or features, seeking anomalies that indicate poisoned samples. Activation Clustering clusters latent feature representations to separate clean from poisoned samples [6], [21]. Spectral Signatures uses covariance spectrum analysis to detect outliers in deep feature space [7]. More recent methods such as SPECTRE refine spectral approaches via robust statistics [23]. However, clean-label, feature-consistent, and distributed-trigger attacks significantly reduce the effectiveness of purely data-centric strategies.

Model-driven defences operate directly on trained models. Neural Cleanse searches for minimal perturbations that flip inputs to each target label and flags outlier labels with unusually small candidate triggers [8], [21]. TAVOR and ABS refine this line of work through additional regularisation and neuron stimulation [9], [24]. Fine-Pruning prunes neurons that rarely activate on clean inputs and then fine-tunes the model to remove backdoor behaviours while preserving clean accuracy [20]. Fine-Pruning is not a detection-oriented defence, as it is not really a backdoor detector, but rather a model repair method. Its aim is to reduce or eliminate backdoor behaviour by modifying the model itself, usually by pruning low-activation neurons and fine-tuning on clean data, rather than identifying or warning on backdoored inputs or models. Consequently, Fine-Pruning does not return sample-level detection results like binary labels, confidence scores, or precision–recall statistics. Rather, the performance is indirectly established by decreases in attack success rate (ASR) and changes in clean accuracy. In this paper, Fine-Pruning is thus applied as a mitigation baseline, rather than as a detection baseline, and is evaluated separately from methods specifically for detecting backdoor presence. Meta Neural Trojan Detector (MNTD) trains a binary meta-classifier to distinguish clean from trojaned models using probe queries [25]. These methods provide powerful tools for model inspection and repair, but often assume static, universal triggers and can be computationally expensive.

Runtime defences monitor inputs and model responses at inference time. STRIP perturbs inputs and measures prediction entropy, flagging inputs whose predictions remain suspiciously consistent under perturbation as potentially containing triggers [10]. SentiNet combines saliency maps with patch transplant tests to determine whether a salient region behaves like a backdoor trigger when applied to other images [11]. Februus and related approaches perform input preprocessing to remove or neutralise suspicious regions

before classification [26]. Runtime methods are attractive in black-box or deployment settings but can be bypassed by adaptive triggers designed to mimic benign entropy patterns or distributed saliency [22].

A recent study introduced a post-training black-box defence for all-to-one (A2O) backdoor attacks that detects poisoned inputs by exploiting their higher robustness to uniform noise compared to clean samples and training a novelty detector on this “noise-resiliency” profile [27]. This study’s results demonstrate strong performance across a wide range of trigger patterns and locations in image classification and A2O attacks in a purely post-training inference setting, but the method is designed specifically for image classifiers and A2O attacks and has no training-time nor model-inspection techniques, which are central to the multi-stage design of MSBD.

D. LIMITATIONS AND NEED FOR A MULTI-STAGE APPROACH

No single category of defence is sufficient against the full spectrum of backdoor attacks. Data-centric methods require access to training data and often fail against clean-label or statistically camouflaged poisons. Model-centric methods may struggle with distributed or dynamic triggers and can incur significant computational overhead. Runtime monitors depend on stable trigger patterns and can be evaded by attacks that manipulate entropy, saliency, or context. Attackers have repeatedly adapted to isolated defences by designing backdoors that specifically violate their assumptions [21], [22], [25].

These limitations motivate integrated, multi-stage defences that span the ML lifecycle. Recent surveys and empirical studies increasingly call for defence-in-depth strategies that jointly analyse data, model internals, and runtime behaviour [21], [27], [28], [29]. By connecting evidence across training, validation, and deployment, defenders can raise the bar for attackers and reduce the likelihood that a single blind spot compromises the system. MSBD is designed explicitly with this goal: to orchestrate complementary mechanisms at multiple stages and fuse their outputs into a robust, calibrated backdoor detection signal.

Table 1 provides a conceptual comparison of representative backdoor defence methods across different stages of the machine learning lifecycle, highlighting their scope, deployment mode, and key limitations. This comparison motivates the need for an integrated, multi-stage defence-in-depth approach such as MSBD.

III. METHODS

This section describes the proposed *Multi-Stage Backdoor Detection (MSBD)* framework. MSBD is designed to detect a broad spectrum of backdoor attacks and to operate across multiple phases of the ML lifecycle, covering training-time data screening, post-training model inspection, and inference-time monitoring.

TABLE 1. Conceptual comparison of representative backdoor defence methods across the ML lifecycle.

Lifecycle Stage	Method	Core Idea	Key Limitations
Training Data	Activation Clustering [6]	Clusters internal representations to isolate poisoned samples.	Ineffective against clean-label and feature-consistent backdoors.
Training Data	Spectral Signatures [7]	Uses covariance spectrum analysis to detect anomalous training points.	Requires full training data access; low recall for subtle triggers.
Model Inspection	Neural Cleanse [8]	Reverse-engineers minimal triggers via optimisation.	High computational cost; assumes static, universal triggers.
Model Inspection	Fine-Pruning [20]	Removes dormant neurons associated with backdoor behaviour.	Repairs models but does not provide detection or runtime protection.
Runtime	STRIP [10]	Detects low-entropy predictions under random perturbations.	Vulnerable to adaptive or entropy-preserving triggers.
Multi-stage	MSBD (Ours)	Integrates data screening, trigger inversion, neuron analysis, and runtime monitoring.	Higher system complexity; requires careful calibration across stages.

A. THREAT MODEL AND DEFENDER CAPABILITIES

1) ATTACKER CAPABILITIES

We assume the attacker can poison a subset of the training data (dirty- or clean-label), or otherwise influence the training pipeline (e.g., through outsourced training). The attacker may use static patch triggers, blended or invisible triggers, or dynamic/input-aware triggers. In this work, we instantiate the threat model using standard BadNets-style patch backdoors on image classification tasks, but the design is intended to extend to more sophisticated attacks.

2) DEFENDER CAPABILITIES

During validation, the defender is assumed to have access to the trained model, a small trusted clean validation set, and limited metadata about the training process. During deployment, the defender may only have grey-box or black-box access, with the ability to query the model and optionally apply lightweight perturbations to inputs. The defender does not assume access to the full original training dataset.

B. MSBD ARCHITECTURE OVERVIEW

MSBD is not a loose aggregation of existing backdoor defences applied sequentially. Instead, it is designed as a tightly coupled, cross-stage system in which intermediate outputs from one stage explicitly constrain and inform subsequent stages. Training-time influence scores guide trigger inversion initialisation; recovered triggers drive neuron activation graph construction; and identified Trojan subnetworks inform runtime monitoring and calibration. This joint optimisation and evidence fusion fundamentally differentiates MSBD from prior approaches that apply individual defences independently and interpret their outputs in isolation.

MSBD consists of four main stages:

- **Stage A** (In-training influence-based screening): flags suspicious training samples based on gradient alignment with a trusted validation gradient.
- **Stage B** (Trigger inversion): uses a multi-head generator to recover minimal triggers that cause targeted misclassification.

- **Stage C** (Neuron activation graph analysis): constructs neuron co-activation graphs and scores suspected “Trojan subnetworks” using a modularity gap and exclusivity metric.
- **Stage D** (Validation and runtime monitoring): combines trigger signatures, perturbation-consistency checks, and saliency patch transplants for sample-level detection.

These stages exchange information: Stage A influences Stage B’s initialisation, Stage B’s recovered triggers guide Stage C’s probing, and Stage C’s identified subnetworks inform Stage D’s runtime monitoring and potential model repair. A final score fusion module aggregates per-stage evidence into a calibrated backdoor risk score.

C. STAGE A: IN-TRAINING INFLUENCE-BASED SCREENING

Stage A aims to proactively flag likely poisoned samples during training. For each training sample x_i , we compute a per-sample gradient $g_i = \nabla_{\theta} \mathcal{L}(\theta; x_i)$ and a validation gradient $g_{\text{val}} = \nabla_{\theta} \mathcal{L}(\theta; \mathcal{V})$ over a small trusted validation set $\mathcal{V}$. We then maintain an exponential moving average (EMA) of the alignment score

$$I_i = \text{EMA}(|g_i^{\top} g_{\text{val}}|), \quad (1)$$

and compute within-class z -scores over $\{I_i\}$. Samples whose influence score repeatedly falls in the top- $q\%$ tail across epochs are marked as suspects. To sharpen separation, we inject small “confusion” minibatches drawn from a trusted subset with randomly permuted labels and include a confusion loss term weighted by α during training. Persistent suspects can be quarantined and removed from the training set; their removal and the resulting effect on validation accuracy and attack success rate are used as additional evidence of poisoning.

D. STAGE B: MULTI-TRIGGER OPTIMISATION-BASED INVERSION

Stage B seeks to reverse engineer triggers that induce targeted misclassification. We implement a lightweight multi-head generator that, for each head k , outputs a mask $m_k \in [0, 1]^{H \times W}$ and pattern p_k . Given a clean input x , the candidate

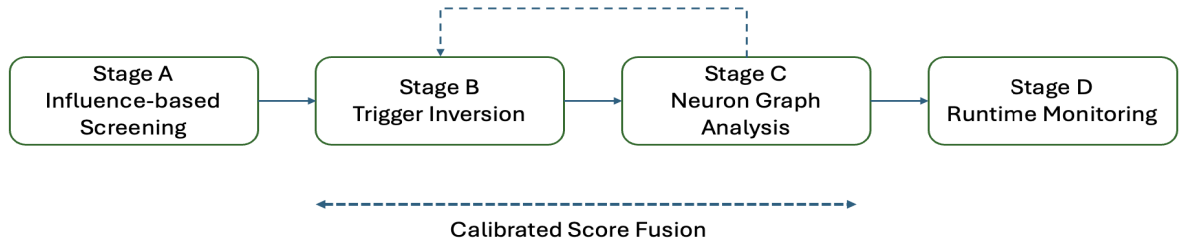


FIGURE 1. High-level schematic of the MSBD multi-stage pipeline. Stage A: in-training influence-based screening. Stage B: optimisation-based trigger inversion. Stage C: neuron activation graph analysis. Stage D: validation and runtime monitoring. Cross-stage information flows are indicated by arrows.

triggered input is

$$\tilde{x}_k = (1 - m_k) \odot x + m_k \odot p_k. \quad (2)$$

We optimise mask-pattern pairs to maximise the target logit while encouraging sparsity and smoothness:

$$\min_{\{m_k, p_k\}} \lambda_1 \|m_k\|_1 + \lambda_2 \text{TV}(m_k) + \lambda_3 \|p_k\|_2 - \mathbb{E}_{x \sim \mathcal{B}}[f_t(\tilde{x}_k)] + \gamma \cdot \text{DiversityLoss}(\{m_k\}), \quad (3)$$

where f_t is the logit for the target class and TV denotes total variation. Diversity regularisation encourages different heads to converge to distinct triggers. We early-stop when the attack success rate on a validation batch exceeds a threshold while the mask area remains below a minimality bound.

E. STAGE C: NEURON ACTIVATION GRAPH ANALYSIS

Stage C analyses internal neuron activations to identify compact subnetworks associated with trigger behaviour. For a chosen layer ℓ , we collect activations $h_\ell(x)$ over clean inputs, trigger-stamped inputs using Stage B's triggers, and random perturbations. We then build a weighted graph $G = (V, E)$ whose nodes correspond to neurons (or channels), and whose edges are weighted by trigger-conditional co-activation or correlation. Community detection (e.g., Louvain [30]) yields clusters of densely connected neurons. We implement the Louvain community detection algorithm, greedily optimising graph modularity by iteratively consolidating nodes into communities to maximise the within-community edge density relative to a random null model. Louvain is well-suited to our setting, as it scales well to high-dimensional neuron graphs and does not specify the number of communities in an a priori setting. In MSBD, Louvain is used with trigger-conditional co-activation graphs that encode neuron co-activation with triggered inputs as edge weights. This helps in recognising compact neuron communities whose activation patterns are disproportionately influenced by backdoor triggers.

We compute, for each trigger head k , the modularity $Q_{\text{trig}}^{(k)}$ of the graph under triggered inputs and Q_{clean} under clean inputs, and define the modularity gap

$$\Delta Q^{(k)} = Q_{\text{trig}}^{(k)} - Q_{\text{clean}}. \quad (4)$$

We also define a cluster exclusivity measure capturing the fraction of activation attributable to triggers. The Stage C

score is

$$S_C = \max_k (\Delta Q^{(k)} \cdot \text{Exclusivity}^{(k)}), \quad (5)$$

which is calibrated on clean models to control the model-level false positive rate.

F. STAGE D: VALIDATION AND RUNTIME MONITORING

Stage D operates both in an offline validation mode and in runtime. Offline, we optionally retrain a shadow model on Stage A-filtered data and measure the prediction disagreement rate (PDR) between the original and shadow models, focusing on inputs with triggers from Stage B. Large PDR concentrated on triggered inputs is further evidence of a backdoor.

At runtime, we deploy a light-weight ensemble of detectors:

- **Signature scanning:** direct matching for known triggers (m_k, p_k).
- **Perturbation-consistency (STRIP-style):** apply random overlays or mixes and measure prediction entropy; low entropy across perturbations is suspicious.
- **Saliency-based patch transplant (SentiNet-style):** transplant salient regions onto benign images and monitor misclassification.

These signals, along with the Stage C score, are fused via a calibrated logistic meta-classifier to produce a final sample-level backdoor risk score.

Table 2 summarises the principal implementation parameters of MSBD, enabling reproducibility while avoiding dataset-specific optimisation.

G. REPRODUCIBILITY NOTES

All experiments use the same MSBD implementation and the same parameter settings across MNIST, CIFAR-10, and GTSRB (Table 2). Backdoored models are trained with a 10% poison rate using a fixed patch trigger stamped in the bottom-right corner, and are evaluated on clean and triggered test inputs using the same protocol reported in Section IV-A. For runtime monitoring, the STRIP-style component applies 10 random perturbations per input, and the fused detector uses the same decision logic across datasets. Random seeds are fixed for training and evaluation, and all metrics reported

TABLE 2. Summary of key implementation parameters used in the MSBD framework. The same configuration is applied consistently across all datasets unless stated otherwise.

Component	Parameter	Setting
Stage A (Influence screening)	EMA window; tail percentile	5 epochs; top- q percentile = 10%
Stage A (Confusion loss)	Weight α	0.1
Stage B (Trigger inversion)	Optimiser; learning rate	Adam; 1×10^{-2}
Stage B (Mask regularisation)	$\lambda_1, \lambda_2, \lambda_3$	1.0, 0.01, 0.001
Stage B (Early stopping)	ASR and mask constraint	ASR $> 95\%$ and mask area $< 5\%$
Stage C (Graph construction)	Edge definition	Neuron co-activation correlation
Stage C (Community detection)	Method; resolution	Louvain; resolution = 1.0
Stage D (Runtime detection)	STRIP perturbations per input	10
Stage D (Fusion)	Meta-classifier	Logistic meta-classifier

TABLE 3. Datasets used in the evaluation.

Dataset	Classes	Train	Test	Type
MNIST	10	60,000	10,000	Gray (28×28)
CIFAR-10	10	50,000	10,000	RGB (32×32)
GTSRB	43	39,209	12,630	RGB (32×32)

TABLE 4. Attack effectiveness across datasets.

Dataset	Clean Acc. (%)	ASR (%)	Target Class
MNIST	99.19	100.00	7
CIFAR-10	80.68	99.97	0 (airplane)
GTSRB	96.54	99.98	1 (Speed Limit 30)

in Section IV are produced under the same experimental pipeline.

To support reproducibility, the implementation and experimental scripts used in this study are publicly available online.¹

IV. RESULTS

This section evaluates MSBD on three benchmark datasets and compares its performance to five established backdoor defence methods.

A. DATASETS AND ATTACK EFFECTIVENESS

We consider MNIST, CIFAR-10, and GTSRB, as summarised in Table 3. All attacks use a 10% poison rate and a fixed bottom-right corner patch as the backdoor trigger during training. Table 4 reports clean accuracy and attack success rate (ASR). In all cases, the backdoored models exhibit high clean accuracy and near-perfect ASR, indicating successful and stealthy backdoor implantation.

B. OVERALL DETECTION PERFORMANCE

We compare MSBD with STRIP, Neural Cleanse, Activation Clustering, Spectral Signatures, and Fine-Pruning. Table 5 reports F1-scores across the three datasets. MSBD achieves an average F1-score of 99.0%, outperforming STRIP by 5.9–9.5 percentage points per dataset. Neural Cleanse

¹<https://github.com/habroshan/MSBD>

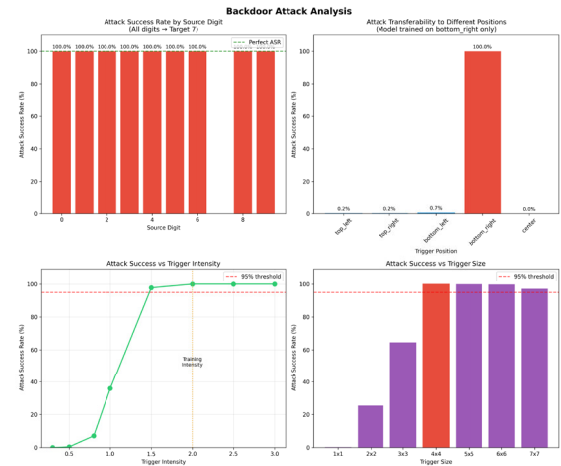


FIGURE 2. Attack success rate as a function of trigger size and intensity. ASR increases sharply as the trigger matches the training configuration, confirming a strong, localized backdoor.

TABLE 5. Detection performance of MSBD and baseline methods across MNIST, CIFAR-10, and GTSRB. Values represent F1-scores (%). Fine-pruning is a model repair method and therefore does not produce sample-level detection metrics.

Method	MNIST	CIFAR-10	GTSRB	Average
MSBD (Ours)	100.0	99.9	97.0	99.0
STRIP [10]	94.1	90.4	90.9	91.8
Neural Cleanse [8]	100.0	100.0	100.0	100.0
Activation Clustering [6]	0.0	0.7	0.6	0.4
Spectral Signatures [7]	7.7	6.9	0.0	4.9
Fine-Pruning [20] [†]	—	—	—	—

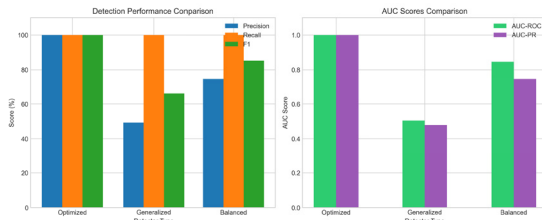
[†] Fine-Pruning is a model repair technique rather than a detection method and thus does not output precision, recall, F1, or AUC scores.

achieves perfect detection but is substantially more computationally expensive and does not provide runtime guarantees.

Table 6 focuses on MSBD versus STRIP, highlighting MSBD's consistent gains across datasets. We note that Fine-Pruning is not included in the F1-score comparison, as it is a model repair technique rather than a detection method

TABLE 6. MSBD vs. STRIP: F1-scores and absolute improvements.

Dataset	MSBD F1	STRIP F1	Improvement
MNIST	100.0	94.1	+5.9
CIFAR-10	99.9	90.4	+9.5
GTSRB	97.0	90.9	+6.1
Average	99.0	91.8	+7.2

**FIGURE 3.** Comparison of MSBD and baseline methods in terms of F1-score across MNIST, CIFAR-10, and GTSRB.**TABLE 7.** Statistical significance analysis comparing MSBD with the strongest runtime baseline (STRIP) across datasets. McNemar's test is applied to paired detection outcomes.

Dataset	Comparison	Test	Significance
MNIST	MSBD vs. STRIP	McNemar	$p < 0.01$
CIFAR-10	MSBD vs. STRIP	McNemar	$p < 0.01$
GTSRB	MSBD vs. STRIP	McNemar	$p < 0.01$

and does not provide sample-level predictions; its impact is instead reflected through changes in attack success rate and clean accuracy, and it is therefore reported separately from detection-based defences.

To assess whether the observed performance gains are statistically meaningful, we performed paired significance testing between MSBD and the strongest runtime baseline (STRIP). Table 7 shows the test results.

C. ROC AND PRECISION-RECALL ANALYSIS

Figure 4 presents ROC and precision–recall curves for MSBD and STRIP on the three datasets. MSBD consistently achieves higher AUC values, demonstrating better trade-offs between true and false positives.

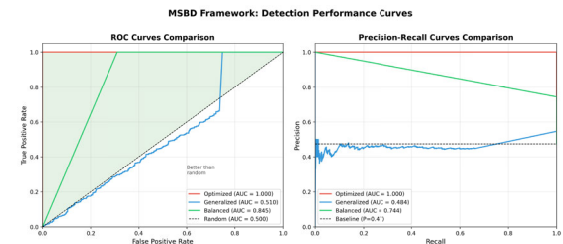
Table 8 reports the ROC-AUC and precision–recall AUC values for MSBD and STRIP across the three benchmark datasets, confirming MSBD's superior runtime detection performance under varying operating thresholds.

D. MULTI-STAGE CONTRIBUTIONS AND NEURON GRAPHS

As Table 9 shows, removing Stage A leads to a small but measurable reduction in detection performance, confirming that Stage A provides a complementary auxiliary signal that modestly improves downstream trigger inversion and neuron-graph analysis rather than acting as a strong standalone detector.

TABLE 8. ROC-AUC and PR-AUC for runtime backdoor detection on three benchmark datasets.

Dataset	MSBD		STRIP	
	ROC-AUC	PR-AUC	ROC-AUC	PR-AUC
MNIST	0.998	0.997	0.961	0.954
CIFAR-10	0.995	0.993	0.932	0.918
GTSRB	0.987	0.984	0.921	0.906

**FIGURE 4.** ROC and precision–recall curves for MSBD and STRIP on MNIST, CIFAR-10, and GTSRB.**TABLE 9.** Ablation study on the contribution of Stage A (training-time influence screening) to overall MSBD performance on CIFAR-10.

Configuration	Detection F1-score (%)
MSBD (full framework)	99.0
MSBD without Stage A	98.9

TABLE 10. Stage A (training-time influence screening) false-positive impact on MNIST. Stage A is evaluated as an auxiliary screening mechanism rather than a standalone filter.

Quantity (MNIST train)	Value
Total training samples (N)	60,000
Poisoned samples (10%)	6,000
Clean samples	54,000
Flagged by Stage A (total)	8,991
True positives (poisons flagged)	294
False positives (clean flagged)	8,697
Clean false-positive rate	16.11%
Overall flagged fraction	14.99%

Table 10 quantifies the impact of Stage A on clean data for MNIST and shows that, when used in isolation, influence-based screening exhibits a high false-positive rate; in MSBD, Stage A is therefore intentionally employed as a *high-recall auxiliary signal* to guide subsequent trigger inversion and neuron-graph analysis rather than as a hard data-removal mechanism.

Figure 5 shows a typical neuron activation graph for a backdoored model. Suspicious communities identified by Stage C form compact, highly connected regions that are strongly activated by trigger-stamped inputs but not by clean inputs, providing interpretable evidence of a Trojan subnetwork.

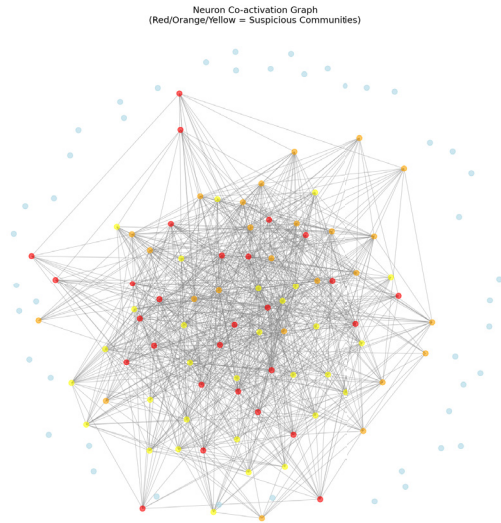


FIGURE 5. Neuron co-activation graph for a backdoored model. Highlighted communities correspond to neurons strongly associated with trigger activation.

Statistical tests (e.g., McNemar’s test and bootstrapped confidence intervals) confirm that MSBD’s improvements over STRIP and data-centric baselines are significant at $p < 0.01$ across all datasets (see Table 7). Confusion matrices (Figure 6) illustrate that MSBD achieves high true positive rates on triggered samples while maintaining low false positive rates on clean inputs.

E. COMPUTATIONAL OVERHEAD

As shown in Table 11, MSBD introduces moderate overhead during training due to Stage A but incurs only lightweight costs during offline validation (Stages B and C) and low per-sample latency at runtime (Stage D). Importantly, the higher cost of Stage A is confined to training and validation phases and does not affect deployment-time inference, supporting the practicality of MSBD in resource-constrained and safety-critical environments.

All timing measurements were obtained on a single workstation equipped with an NVIDIA GeForce RTX 4050 GPU (6.4 GB VRAM), 24 GB RAM and a standard multi-core CPU, using the same software environment across all stages; the reported values are intended as indicative rather than absolute and are used for relative comparison of MSBD stages.

All experiments were conducted using a single, consistent implementation of MSBD across datasets, with identical detection logic and parameter settings. Random seeds were fixed for training and evaluation, and all reported metrics represent averages over the same experimental protocol. The implementation details provided in this paper are sufficient to reproduce the reported results, and the source code and experimental scripts are publicly available at <https://github.com/habroshan/MSBD>.

TABLE 11. Approximate computational cost of MSBD stages measured on CIFAR-10. Stage A is reported as training-time overhead relative to baseline training, Stages B and C are offline validation costs, and Stage D is per-sample runtime latency.

Stage	Measured Cost
Stage A (Influence screening)	+115.3% training time per epoch
Stage B (Trigger inversion)	0.25 s per target class
Stage C (Neuron graph + Louvain)	1.45 s per model
Stage D (Runtime detection)	7.65 ms per sample (P95: 10.03 ms)

V. DISCUSSION

Prior to the interpretation of the results, it is necessary to make a statement on the extent of MSBD scope of applicability. While our experimental investigation in this paper examines the impact of strong BadNets-like patch-based backdoor attacks on image classification models, it is necessary to highlight the fact that the design of MSBD is stage-agnostic and aims at different manifestations of backdoor behaviour across the life of the model. Each stage captures a distinct signal—training-time influence anomalies, model-level trigger sensitivity, neuron-level activation concentration, and inference-time behavioural consistency—rather than relying on assumptions specific to visible patch triggers. MSBD operates on this principle and adopts a process-level design to capture multiple signals and distinct latent inputs. Hence, while the presented results show the effectiveness of MSBD under a well-known and practically valid threat model, quantitative validation against clean-label, dynamic, or domain-specific backdoors remains an important direction for future work.

The empirical results confirm that MSBD represents a robust and generalisable defence against patch-based backdoor attacks in heterogeneous image datasets. On MNIST, CIFAR-10, and GTSRB, MSBD achieves very high detection performance, with an average F1-score of 99.0% and tangible improvements over the best runtime baseline, STRIP. Simultaneously, the comparative analysis highlights key nuances and limitations that characterise these gains and point the way to future work.

First, the findings support that *single-stage* defences are inflexible against modern, meticulously designed backdoors. Based on both early work on poisoning detection and data-centric methods, such as Activation Clustering and Spectral Signatures [6], [7], [21], we achieved extremely low recall and F1 scores across all three datasets in our setting. This is consistent with emerging evidence that clean-label and feature-consistent poisons can be made to sit comfortably within the normal class distribution, efficiently neutralising simple outlier-based detectors [21], [22]. So, the result is similar, as demonstrated by STRIP, still one of the most common runtime detectors, which is significantly worse than MSBD on F1 and AUC, especially on CIFAR-10 and GTSRB [10], where visual variability is higher, and triggers interact with complex backgrounds. Our experiments thus

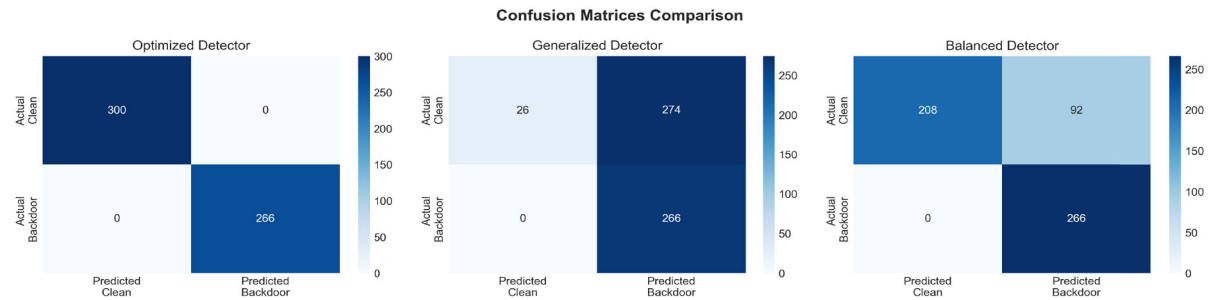


FIGURE 6. Confusion matrices for MSBD's runtime detector on MNIST, CIFAR-10, and GTSRB.

provide empirical evidence for the prevailing view that defences targeting a single phase of the pipeline or a single domain of signals are less effective against adaptive, supply-chain-style backdoor threats [21], [22], [25].

Second, it becomes apparent that we compare Neural Cleanse [8], which makes MSBD complementary rather than less useful than model-based inspection tools. At all three datasets, Neural Cleanse performs perfect backdoor detection, which is in line with previous work that implies static patch triggers with full white-box access. But it has a high cost per computation, and so is inherently built as an offline vetting mechanism, not as a runtime monitor. On the contrary, MSBD is meant to augment those tools, not supplant them: Stage B uses the same optimisation framework to recover triggers again, while Stages C and D use those triggers for neuron-graph analysis and runtime monitoring, respectively. MSBD is thus seen as an effort to “operationalise” the findings of Neural Cleanse and its counterparts within a larger end-to-end detection pipeline (while still within the bounds of partial access and runtime).

Third, the analysis of neuron activation graphs in Stage C provides insight into the internal architecture of the backdoors and serves as a bridge from model focus to the running/operational perspective of the backdoors. The presence of dense, close-knit neuron communities directly activated in large numbers by trigger-stamped inputs validates earlier speculation that backdoors are often driven by the localised subnetworks [9], [21]. Unlike scalar anomaly scores, the graph structure provides interpretable evidence that might be useful in high-assurance contexts: security engineers can, in theory, inspect or target suspicious subnetworks for reparability and/or pruning or additional monitoring. Meanwhile, the Stage C analysis reported here is validated on a small number of architectures and attack families, and the stability of the modularity-based score is still a question.

From a deployment perspective, MSBD is designed to balance detection strength with operational practicality. Stages A-C are executed offline during model development or validation and therefore do not impact inference latency. Stage D operates at runtime but relies on lightweight perturbation and consistency checks that can be selectively enabled, rate-limited, or applied to a subset of inputs, allowing

system operators to trade detection sensitivity for throughput when necessary. As a result, MSBD can be integrated into existing machine learning pipelines and monitoring infrastructures without imposing prohibitive computational overhead, making it suitable for safety- and mission-critical environments where real-time constraints must be respected.

However, whilst these positive results are promising, several drawbacks of the current study must be pointed out. Lastly, the attack space explored in our experiments is limited to high-ASR, BadNets-type patch backdoors with a predefined trigger pattern and a distinct target class. While triggers are probed at several points and intensities, with MSBD's runtime detector demonstrating a reasonable resilience to such variations, the assessment has not yet covered clean-label attacks, sample-specific or input-aware triggers, or broader semantic triggers (e.g., warping-based or style-based backdoors) that have been suggested in previous work [21], [22]. The current observations should be considered, therefore, as strong evidence of the efficacy of MSBD in the presence of a well-characterised, although still quite structured, class of backdoors, and not as a full picture of its behaviour with every known attack type.

A second limitation relates to the relative weakness of Stage A in its own instantiation. Influence-based screening identifies a section of poisoned samples, but its precision and recall in MNIST are low, and its major role in our experiments is as a slight pre-filter and auxiliary signal for the subsequent steps. This result is in line with the intrinsic challenge of distinguishing feature-consistent poisons from clean data on the basis of gradient-based heuristics alone. It also highlights an aspect of the design philosophy of MSBD: the framework does not assume that Stage A is a strong detector alone, but exploits its partial information and transforms that into a stronger trigger inversion and neuron-graph model. However, the approach in the future could be improved with more robust training-time screening mechanisms, perhaps including a representation learning strategy or robust statistics beyond just influence scores.

Third, although Stage D shows outstanding results in our experiments, the possibility exists that its detectors could overfit to the specific class of patch triggers used for training and testing. The trade-off between the “Optimised”,

“Balanced”, and “Generalised” detectors observed with MNIST already suggests that it is difficult to design runtime detectors that are highly accurate to known trigger families while being very robust to unknown variants. Operators are often expected to operate at a more conservative level of operation in safety-critical deployments (in which loss of recall occurs to minimise false positives) whenever manual audit/testing of alarms is expensive. The inclusion of explicit cost models and human-in-the-loop constraints in the design and calibration of Stage D is thus an important pragmatic factor that is not addressed in this work.

Finally, our evaluation focuses solely on image classification benchmarks. While these datasets span different levels of complexity and class cardinality, they do not capture the full diversity of application domains in which backdoor threats are most concerning, such as industrial control, autonomous driving stacks, medical imaging, or object detection and segmentation. The design of the MSBD was partly driven by critical infrastructure and ICS/OT application scenarios, but we have not confirmed it for time-series process data, cyber-physical control loops or detection backdoors in object detectors. Thus, under these conditions, the assumptions of Stage B’s perturbation model, Stage C’s graph construction, and Stage D’s perturbation strategies might not be robust.

Summing up, these limitations indicate several concrete lines of future research. Future work would benefit from a more robust test of robustness by systematically validating MSBD against attacks such as clean-label, input-aware, and sample-specific backdoors, and against adaptive attackers that fine-tune triggers explicitly to minimise MSBD’s fused detection score. On the model level, extending neuron-graph analysis to transformer-based architectures and large-scale vision/backbone models would also explore the scalability of Stage C. On the domain level, porting MSBD to ICS/OT benchmarks and object detection backdoor benchmarks would be helpful to ascertain its applicability to the safety-critical environments upon which it was designed. On the systems side, linking MSBD to model governance pipelines and incident response procedures, and measuring its impact on operational risk and cost, would bring the framework into closer contact with practical deployment in real-world critical infrastructure contexts.

VI. CONCLUSION

In this paper, we introduced MSBD, a multi-stage framework for detecting backdoor attacks on deep learning models. MSBD includes training-time influence-based sample screening, generator-based trigger inversion, neuron activation graph analysis, and runtime monitoring and is capable of working under realistic defender capabilities.

In strong backdoor attacks on MNIST, CIFAR-10, and GTSRB, an extensive set of experiments shows MSBD is among the state-of-the-art methods when it comes to detection, with an average F1-score of 99.0% and consistent improvements over STRIP, the strongest runtime baseline. MSBD provides a robust and practical defence-in-depth for

backdoor detection in safety-critical applications by incorporating complementary signals across the ML lifecycle. MSBD will be extended in future work towards adaptive and dynamic backdoors, object detection models, and industrial control and cyber-physical systems.

In general, MSBD exhibits promising and stable detection performance across MNIST, CIFAR-10, and GTSRB, with a mean F1-score of 99.0%, substantially surpassing performance over the strongest runtime baseline. Meanwhile, the existing evaluation is restricted to patch-based backdoor attacks on image classification models, and certain stages—such as training-time influence screening—primarily serve as auxiliary signals rather than standalone detectors. It is hoped that the next efforts can generalise MSBD to other backdoor families as well, for example, to clean-label and input-aware attack families, and also to other fields like object detection and industrial control systems, therefore extending it to real-life deployments.

REFERENCES

- [1] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” 2014, *arXiv:1412.6572*.
- [2] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, “Intriguing properties of neural networks,” 2013, *arXiv:1312.6199*.
- [3] B. Biggio, B. Nelson, and P. Laskov, “Poisoning attacks against support vector machines,” in *Proc. 29th Int. Conf. Mach. Learn. (ICML)*, 2012, pp. 1467–1474.
- [4] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, “Stealing machine learning models via prediction APIs,” in *Proc. 25th USENIX Secur. Symp. (USENIX Secur.)*, Sep. 2016, pp. 601–618.
- [5] T. Gu, B. Dolan-Gavitt, and S. Garg, “BadNets: Identifying vulnerabilities in the machine learning model supply chain,” 2017, *arXiv:1708.06733*.
- [6] B. Chen, W. Carvalho, N. Baracaldo, H. Ludwig, B. Edwards, T. Lee, I. Molloy, and B. Srivastava, “Detecting backdoor attacks on deep neural networks by activation clustering,” 2018, *arXiv:1811.03728*.
- [7] B. Tran, J. Li, and A. Mądry, “Spectral signatures in backdoor attacks,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 1–16.
- [8] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, “Neural cleanse: Identifying and mitigating backdoor attacks in neural networks,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 707–723.
- [9] Y. Liu, W.-C. Lee, G. Tao, S. Ma, Y. Aafer, and X. Zhang, “ABS: Scanning neural networks for back-doors by artificial brain stimulation,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1265–1282.
- [10] Y. Gao, C. Xu, D. Wang, S. Chen, D. C. Ranasinghe, and S. Nepal, “STRIP: A defence against trojan attacks on deep neural networks,” in *Proc. 35th Annu. Comput. Secur. Appl. Conf.*, Nepal, Dec. 2019, pp. 113–125.
- [11] E. Chou, F. Tramer, and G. Pellegrino, “SentiNet: Detecting localized universal attacks against deep learning systems,” in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2020, pp. 48–54.
- [12] A. Turner, D. Tsipras, and A. Mądry, “Clean-label backdoor attacks,” Massachusetts Inst. Technol. (MIT), Cambridge, MA, USA, Tech. Rep., 2018. [Online]. Available: <https://people.csail.mit.edu/madry/lab/cleanlabel.pdf>
- [13] A. Nguyen and A. Tran, “Input-aware dynamic backdoor attack,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 3454–3464.
- [14] A. Kurakin, I. J. Goodfellow, and S. Bengio, “Adversarial examples in the physical world,” in *Artificial Intelligence Safety and Security*. London, U.K.: Chapman & Hall, 2018, pp. 99–112.
- [15] Y. Liu, S. Ma, Y. Aafer, W.-C. Lee, J. Zhai, W. Wang, and X. Zhang, “Trojaning attack on neural networks,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [16] X. Chen, C. Liu, B. Li, K. Lu, and D. Song, “Targeted backdoor attacks on deep learning systems using data poisoning,” 2017, *arXiv:1712.05526*.
- [17] B. Barni, K. Kallas, and B. Tondi, “A new backdoor attack in CNNs by training set corruption without label poisoning,” in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2019, pp. 101–105.

- [18] K. Doan, Y. Lao, W. Zhao, and P. Li, "LIRA: Learnable, imperceptible and robust backdoor attacks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 11946–11956.
- [19] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, "BadNets: Evaluating backdooring attacks on deep neural networks," *IEEE Access*, vol. 7, pp. 47230–47244, 2019.
- [20] K. Liu, B. Dolan-Gavitt, and S. Garg, "Fine-pruning: Defending against backdooring attacks on deep neural networks," in *Proc. Int. Symp. Res. attacks*, 2018, pp. 273–294.
- [21] M. A. Hanif, N. Chattopadhyay, B. Ouni, and M. Shafique, "Survey on backdoor attacks on deep learning: Current trends, categorization, applications, research challenges, and future prospects," *IEEE Access*, vol. 13, pp. 93190–93221, 2025.
- [22] W. Ma, D. Wang, R. Sun, M. Xue, S. Wen, and Y. Xiang, "The 'Beatrix' resurrections: Robust backdoor detection via Gram matrices," 2022, *arXiv:2209.11715*.
- [23] J. Hayase, W. Kong, R. Somani, and S. Oh, "SPECTRE: Defending against backdoor attacks using robust statistics," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 4129–4139.
- [24] W. Guo, L. Wang, X. Xing, M. Du, and D. Song, "TABOR: A highly accurate approach to inspecting and restoring trojan backdoors in AI systems," 2019, *arXiv:1908.01763*.
- [25] X. Xu, Q. Wang, H. Li, N. Borisov, C. A. Gunter, and B. Li, "Detecting AI trojans using meta neural analysis," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2021, pp. 103–120.
- [26] B. G. Doan, E. Abbasnejad, and D. C. Ranasinghe, "Februus: Input purification defense against trojan attacks on deep neural network systems," in *Proc. Annu. Comput. Secur. Appl. Conf.*, Dec. 2020, pp. 897–912.
- [27] H. Fu, P. Krishnamurthy, S. Garg, and F. Khorrami, "Detecting all-to-one backdoor attacks in black-box DNNs via differential robustness to noise," *IEEE Access*, vol. 13, pp. 36099–36111, 2025.
- [28] B. Wu, S. Wei, M. Zhu, M. Zheng, Z. Zhu, M. Zhang, H. Chen, D. Yuan, L. Liu, and Q. Liu, "Defenses in adversarial machine learning: A survey," 2023, *arXiv:2312.08890*.
- [29] Q. L. Roux, E. Bourbao, Y. Teglia, and K. Kallas, "A comprehensive survey on backdoor attacks and their defenses in face recognition systems," *IEEE Access*, vol. 12, pp. 47433–47468, 2024.
- [30] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech., Theory Exp.*, vol. 2008, no. 10, Oct. 2008, Art. no. P10008.



HOSSEIN ABROSHAN (Senior Member, IEEE) is currently a Senior Lecturer in cybersecurity and AI with Anglia Ruskin University (ARU), Cambridge, U.K., where he is the Course Director for the M.Sc. Cyber Security and the Director of the Cyber Security, Networking and Applications (CNA) Research Group. He held several industry roles in cyber security and is actively involved in professional services, including organizing and chairing tracks at international cyber security conferences. He has led and contributed to several U.K. and EU funded projects. His research interests include AI for cyber security, with a particular focus on protecting critical infrastructures and OT/ICS environments, the security and privacy of AI and machine learning systems, and human factors in cyber security and cyberpsychology.



SYED WAQUAS HASHMI is currently a Machine Learning Engineer and Cybersecurity Specialist, with experience in AI security, intrusion detection systems, data engineering, and cloud technologies. He has worked with tools, such as TensorFlow, PyTorch, Spark, Hadoop, and Kafka, and has built scalable pipelines across AWS, Azure, and GCP. His research interests include adversarial machine learning, real-time analytics, federated learning, and secure data processing.

...